nccl-tests 2-node summary (multi-collective)

- Generated: 2026-08-06 18:44:04
- Runs: node5500+node5501, node5500+node5502, node5501+node5502
- Note: node5500-5502 are no longer in Slurm. The tables below are their historical baseline; the current hardware is the 7 new nodes covered in "New nodes" below.
- GPUs: 8/node x 2 nodes = 16 x NVIDIA B200 (inter-node, InfiniBand + GPUDirect RDMA)
- Config: 1 MiB-16 GiB, 5 warmup + 20 iters
- Reference: MIT aicr-benchmarks `results_b200.md` Table 2 (b0029+b0030, 16x B200 / NDR IB). busbw is the figure of merit.

Per-collective busbw vs B200 reference

Representative node pair: **node5500+node5501**.

Collective	GPUs	converged busbw (GB/s)	peak busbw (GB/s)	reference busbw (GB/s)	ours / ref	HW max (GB/s)	ours / HW max	correctness
sendrecv	16	49.7	50.0	26.6	—	50	99%	PASS
all_reduce	16	239.9	239.9	170.0	141%	400	60%	PASS
all_gather	16	366.8	366.8	218.0	168%	400	92%	PASS
reduce_scatter	16	375.2	375.2	218.0	172%	400	94%	PASS
reduce	16	368.6	368.6	201.0	183%	400	92%	PASS
broadcast	16	368.1	368.1	202.0	182%	400	92%	PASS
alltoall	16	47.5	48.4	39.8	119%	400	12%	PASS
gather	16	95.4	95.4	90.5	105%	400	24%	PASS
scatter	16	325.2	325.2	293.0	111%	400	81%	PASS

Converged = busbw at the largest message size, best of out-of-place / in-place (matches the reference methodology).

The other node pair(s) — node5500+node5502, node5501+node5502 — give essentially identical results and are omitted here to keep the table readable: across every collective the largest deviation from node5500+node5501 is **5.0%** (alltoall on node5500+node5502). No pair stands out as slow, so the fabric behaves the same whichever two of the three nodes are used. Per-pair message-size detail for all pairs is in the

next section.

`HW max` is the hardware ceiling of **this** cluster's fabric, not a figure taken from any paper. Each B200 owns one NDR rail at 400 Gb/s = **50 GB/s per direction**, and each node has **8 rails** (mlx5_4/7/8/9/10/13/14/15, confirmed by `ibstat`), so:

- **sendrecv** — busbw is one pair's rate => ceiling **50 GB/s**.
- **all other collectives** — ring/symmetric or root-anchored, driving all 8 rails concurrently => ceiling 8 x 50 = **400 GB/s** per node per direction.

The NIC is the binding constraint in both directions because PCIe Gen5 x16 is full-duplex (~63 GB/s *each* way), comfortably above the 50 GB/s rail. A collective well below its ceiling is limited by the NCCL algorithm, not by this hardware.

New nodes (node5600/5601/5602/5702/5800/5801/5802)

Run 2026-08-24 on 5 pairs of the 7 new nodes, same configuration as above

(all collectives, 8 GPUs/node = 16 ranks, 1 MiB-16 GiB), plus one confirmation

pair run afterward to isolate the cause of an anomaly (see below).

**4 of 6 pairs tested match the old nodes. Two do not, and the cause is a

specific route, not a bad node.**

Collective	old node5500 +5501	healthy new pairs (mean, n=4)	vs old	node5602/5702 + node5802-c1 (mean, n=2)	vs healthy
sendrecv	49.7	48.8	-1.9%	48.8	+0.1%
all_reduce	239.9	233.1	-2.8%	235.4	+1.0%
all_gather	366.8	375.4	+2.3%	**194.2**	**-48.3%**
reduce_scatter	375.2	382.4	+1.9%	**200.9**	**-47.5%**
reduce	368.6	374.8	+1.7%	**198.1**	**-47.1%**
broadcast	368.1	359.1	-2.4%	**198.8**	**-44.7%**
alltoall	47.5	48.9	+2.9%	44.0	-10.1%
gather	95.4	93.0	-2.5%	94.1	+1.2%
scatter	325.2	335.9	+3.3%	310.1	-7.7%

Healthy pairs = node5600-c1+node5601-c1, node5602-c1+node5702-c1,

node5800-c1+node5801-c1, and **node5800-c1+node5802-c1** (confirmation run,

below). Every collective on those is within **3.3%** of the old node5500+node5501

figures, so the analysis and the ceilings in this document carry over unchanged.

**The problem: the four ring collectives run at almost exactly half bandwidth,
but only on the routes node5602-c1<->node5802-c1 and node5702-c1<->node5802-c1**
(~198 GB/s against ~375-380 GB/s, i.e. 4 rails' worth of traffic instead of 8).
The first hypothesis was "node5802-c1 is degraded" — ruled out by a targeted
confirmation run pairing it with a *known-good* partner instead:

Pair	reduce_scatter	all_gather
node5800-c1 + node5801-c1 (baseline)	384.5	370.6
node5800-c1 + node5802-c1 (confirmation)	**380.5**	**377.0**
node5602-c1 + node5802-c1 (original)	204.8	191.7
node5702-c1 + node5802-c1 (original)	196.9	196.8

**node5802-c1 is fine — full bandwidth with node5800-c1 (same 58xx chassis
group).** The degradation appears only in its cross-chassis routes to node5602-c1
(56xx) and node5702-c1 (57xx), while node5602-c1<->node5702-c1 (also cross-group)
is itself fully healthy. So this is neither "one bad node" nor "crossing chassis
groups is slow" — it isolates to the fabric path(s) connecting node5802-c1
specifically to the 56xx/57xx switches, most likely one or more degraded links or
a routing/ECMP imbalance on that particular leaf-to-spine hop. What further rules
out a node- or driver-level cause: all 8 rails report `Active`/400 Gb/s on every
node; node5802-c1 scores normally on gpu-fryer and every 1-node NVLink
collective; `nvidia_peermem`, driver version, and subnet manager LID all match a
healthy node; both slow runs completed with PASS correctness and zero
`NCCL_DEBUG=WARN` output; and `sendrecv`/`all_reduce` (which don't depend on all
8 rails being open on the same route) are unaffected on the same pairs.
Recommended follow-up for ORCD: check fabric routing/cabling specifically on
the node5802-c1 <-> 56xx and node5802-c1 <-> 57xx paths (leaf/spine assignment,
per-link counters for CRC errors or symbol errors) rather than on node5802-c1
itself. A re-run with `NCCL_DEBUG=INFO` on the two affected pairs would show how
many channels NCCL opens across the node boundary and pin down which of the 8
rails is the bottleneck.

Interpreting `ours / HW max`

Dividing each result by one rail's line rate (50 GB/s) gives the most useful view: **how many of the 8 rails the collective actually engages**.

Collective	ours / HW max	effective rails (of 8)	verdict
sendrecv	99%	0.99 (per pair)	at line rate
reduce_scatter	94%	7.50	at fabric limit
reduce	92%	7.37	at fabric limit
broadcast	92%	7.36	at fabric limit
all_gather	92%	7.34	at fabric limit
scatter	81%	6.50	expected (root-anchored)
all_reduce	60%	4.80	expected Ring two-pass penalty
gather	24%	1.91	NCCL algorithm limit
alltoall	12%	0.95	NCCL algorithm limit

At the hardware limit (92-99%). `sendrecv` is the cleanest validation in the table: each GPU saturates its own rail, so ~99% of 50 GB/s means nothing is left on the table. It is the single number that certifies the fabric is healthy. The ring collectives sit at 7.3-7.5 effective rails because NCCL runs 8 parallel ring channels, each crossing the node boundary on a different rail; the missing few percent is ring fill/drain and protocol overhead, which cannot be recovered.

Expected shortfalls. `all_reduce` at ~60% is the Ring two-pass penalty: it runs reduce_scatter then all_gather, and the busbw formula already divides out the doubled traffic (factor 2(N-1)/N), so a perfectly pipelined all_reduce would score the *same* as all_gather. It does not, because the ring fills and drains twice and pays the phase-transition latency. That fixed latency does not shrink when bandwidth grows, which is why our all_reduce is a *smaller* fraction of our all_gather (~65%) than the reference's was (~78%) — and why SHARP, which collapses the two passes into one in-switch reduction, has more to gain here (see `out-nccl-2node-sharp/`). `scatter` is root-anchored and unidirectional, limited by the root's own outbound capacity.

Algorithm-limited, and the numbers say so precisely. `alltoall` at ~12% is exactly 1/8 — it engages roughly **one rail's worth** of bandwidth out of 8, a literal quantification of NCCL's N^2 point-to-point transfers not being pipelined across NICs. `gather` at ~24% is about two rails, the same story for fan-in to a single root. The decisive evidence that these are algorithmic rather than physical: this fabric is ~1.9x faster than the reference on sendrecv, yet gather improved only 1.05x and alltoall 1.19x. A faster fabric barely helps a collective that is not using it.

> **Caveat on the denominators.** The 400 GB/s ceiling is exact for the ring collectives, whose traffic streams around a ring bottlenecked by its inter-node links. It is an *approximation* for the root-anchored and all-to-all patterns, where only a fraction of traffic crosses the node boundary (for alltoall, 8 of each GPU's 15 peers are remote; the rest go over NVLink). A per-collective ceiling would shift those percentages — most likely lowering

`scatter`'s apparent figure. It does not change any conclusion: gather and alltoall are 4-8x below any reasonable ceiling and are algorithm-bound under every accounting.

> Note: sendrecv here uses 16 GPUs (ring), but the reference 26.6 GB/s is a per-pair (2-GPU) bidir figure, so the two are not directly comparable and `ours / ref` is left blank.

Bus bandwidth vs message size (GB/s)

node5500+node5501 — sendrecv

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	52.5 us	20.0	55.7 us	18.8
4 MiB	102.1 us	41.1	98.5 us	42.6
16 MiB	346.1 us	48.5	346.9 us	48.4
64 MiB	1.36 ms	49.4	1.36 ms	49.3
256 MiB	5.40 ms	49.7	5.39 ms	49.8
1 GiB	21.52 ms	49.9	21.52 ms	49.9
4 GiB	85.89 ms	50.0	85.84 ms	50.0
16 GiB	345.81 ms	49.7	359.20 ms	47.8

node5500+node5501 — all_reduce

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	568.7 us	3.5	472.9 us	4.2
4 MiB	1.10 ms	7.2	879.3 us	8.9
16 MiB	884.5 us	35.6	929.3 us	33.9
64 MiB	2.11 ms	59.6	2.10 ms	60.0
256 MiB	3.43 ms	146.6	3.73 ms	134.8
1 GiB	12.55 ms	160.4	11.46 ms	175.6
4 GiB	37.19 ms	216.6	36.50 ms	220.6
16 GiB	134.29 ms	239.9	135.65 ms	237.5

node5500+node5501 — all_gather

Message size	OOP time	OOP busbw	IP time	IP busbw

1 MiB	584.8 us	1.7	632.9 us	1.6
4 MiB	700.7 us	5.6	684.1 us	5.8
16 MiB	615.1 us	25.6	557.8 us	28.2
64 MiB	1.28 ms	49.1	1.26 ms	49.8
256 MiB	1.33 ms	188.6	1.34 ms	187.2
1 GiB	3.37 ms	298.3	3.50 ms	288.0
4 GiB	12.21 ms	329.8	11.88 ms	338.9
16 GiB	45.08 ms	357.2	43.91 ms	366.8

node5500+node5501 — reduce_scatter

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	491.0 us	2.0	715.0 us	1.4
4 MiB	539.0 us	7.3	529.5 us	7.4
16 MiB	455.5 us	34.5	451.3 us	34.9
64 MiB	1.22 ms	51.6	1.20 ms	52.6
256 MiB	1.42 ms	177.8	1.32 ms	190.9
1 GiB	3.18 ms	316.1	3.20 ms	314.3
4 GiB	11.91 ms	338.1	10.99 ms	366.3
16 GiB	43.64 ms	369.1	42.92 ms	375.2

node5500+node5501 — reduce

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	580.0 us	1.8	576.9 us	1.8
4 MiB	154.8 us	27.1	99.0 us	42.4
16 MiB	168.4 us	99.6	170.2 us	98.6
64 MiB	356.4 us	188.3	354.3 us	189.4
256 MiB	1.12 ms	239.8	1.11 ms	241.6
1 GiB	4.12 ms	260.6	4.12 ms	260.6
4 GiB	13.15 ms	326.5	13.08 ms	328.2
16 GiB	46.84 ms	366.8	46.60 ms	368.6

node5500+node5501 — broadcast

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	382.2 us	2.7	438.4 us	2.4
4 MiB	332.9 us	12.6	342.4 us	12.2
16 MiB	392.4 us	42.8	326.7 us	51.4
64 MiB	506.5 us	132.5	509.6 us	131.7
256 MiB	1.37 ms	196.2	1.37 ms	195.5
1 GiB	4.31 ms	249.2	4.42 ms	243.1
4 GiB	13.74 ms	312.6	12.82 ms	334.9
16 GiB	46.68 ms	368.0	46.66 ms	368.1

node5500+node5501 — alltoall

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	712.8 us	1.4	919.3 us	1.1
4 MiB	836.8 us	4.7	774.9 us	5.1
16 MiB	1.33 ms	11.8	1.21 ms	13.0
64 MiB	3.31 ms	19.0	3.09 ms	20.4
256 MiB	8.86 ms	28.4	7.83 ms	32.1
1 GiB	25.74 ms	39.1	22.81 ms	44.1
4 GiB	83.20 ms	48.4	88.63 ms	45.4
16 GiB	339.09 ms	47.5	358.11 ms	45.0

node5500+node5501 — gather

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	51.6 us	19.0	52.3 us	18.8
4 MiB	68.2 us	57.6	70.5 us	55.8
16 MiB	178.8 us	88.0	177.4 us	88.7
64 MiB	680.5 us	92.5	680.1 us	92.5

256 MiB	2.73 ms	92.1	2.73 ms	92.1
1 GiB	10.94 ms	92.0	10.94 ms	92.0
4 GiB	43.78 ms	92.0	43.78 ms	92.0
16 GiB	175.11 ms	92.0	168.83 ms	95.4

node5500+node5501 — scatter

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	343.8 us	2.9	212.2 us	4.6
4 MiB	234.9 us	16.7	312.7 us	12.6
16 MiB	214.3 us	73.4	153.2 us	102.6
64 MiB	297.0 us	211.8	300.5 us	209.3
256 MiB	1.21 ms	207.4	1.24 ms	203.7
1 GiB	3.92 ms	256.8	3.53 ms	284.8
4 GiB	13.39 ms	300.7	12.61 ms	319.4
16 GiB	49.70 ms	324.1	49.52 ms	325.2

node5500+node5502 — sendrecv

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	54.4 us	19.3	52.8 us	19.9
4 MiB	102.3 us	41.0	100.6 us	41.7
16 MiB	355.1 us	47.2	352.3 us	47.6
64 MiB	1.39 ms	48.4	1.39 ms	48.2
256 MiB	5.51 ms	48.7	5.51 ms	48.7
1 GiB	21.97 ms	48.9	21.96 ms	48.9
4 GiB	92.66 ms	46.4	92.69 ms	46.3
16 GiB	360.23 ms	47.7	355.18 ms	48.4

node5500+node5502 — all_reduce

Message size	OOP time	OOP busbw	IP time	IP busbw

1 MiB	383.6 us	5.1	302.3 us	6.5
4 MiB	1.05 ms	7.5	894.1 us	8.8
16 MiB	1.12 ms	28.2	1.07 ms	29.5
64 MiB	2.37 ms	53.0	2.36 ms	53.4
256 MiB	4.65 ms	108.2	4.82 ms	104.5
1 GiB	12.87 ms	156.4	12.63 ms	159.4
4 GiB	39.65 ms	203.1	38.02 ms	211.8
16 GiB	140.81 ms	228.8	138.11 ms	233.2

node5500+node5502 — all_gather

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	580.9 us	1.7	482.2 us	2.0
4 MiB	507.1 us	7.8	536.4 us	7.3
16 MiB	543.0 us	29.0	454.9 us	34.6
64 MiB	1.31 ms	48.0	1.33 ms	47.5
256 MiB	1.51 ms	166.8	1.48 ms	170.0
1 GiB	3.25 ms	310.0	3.20 ms	314.6
4 GiB	11.64 ms	345.8	11.12 ms	362.2
16 GiB	42.75 ms	376.7	42.09 ms	382.7

node5500+node5502 — reduce_scatter

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	504.8 us	1.9	443.6 us	2.2
4 MiB	532.6 us	7.4	550.5 us	7.1
16 MiB	572.4 us	27.5	645.8 us	24.4
64 MiB	1.30 ms	48.3	1.27 ms	49.5
256 MiB	1.38 ms	182.5	1.33 ms	189.6
1 GiB	3.33 ms	302.1	3.29 ms	305.6
4 GiB	11.26 ms	357.8	11.16 ms	360.9
16 GiB	42.75 ms	376.8	42.12 ms	382.4

node5500+node5502 — reduce

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	514.4 us	2.0	470.3 us	2.2
4 MiB	259.3 us	16.2	259.3 us	16.2
16 MiB	309.4 us	54.2	312.2 us	53.7
64 MiB	481.5 us	139.4	469.7 us	142.9
256 MiB	1.22 ms	220.3	1.21 ms	222.2
1 GiB	3.97 ms	270.5	4.07 ms	263.6
4 GiB	12.92 ms	332.5	12.80 ms	335.4
16 GiB	46.11 ms	372.6	44.89 ms	382.7

node5500+node5502 — broadcast

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	452.8 us	2.3	518.1 us	2.0
4 MiB	204.1 us	20.6	113.0 us	37.1
16 MiB	191.3 us	87.7	191.9 us	87.4
64 MiB	419.1 us	160.1	416.6 us	161.1
256 MiB	1.40 ms	192.2	1.41 ms	191.0
1 GiB	4.78 ms	224.7	4.76 ms	225.5
4 GiB	13.74 ms	312.6	13.48 ms	318.6
16 GiB	47.72 ms	360.0	47.47 ms	361.9

node5500+node5502 — alltoall

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	860.9 us	1.1	817.2 us	1.2
4 MiB	973.5 us	4.0	759.5 us	5.2
16 MiB	2.02 ms	7.8	941.0 us	16.7
64 MiB	2.94 ms	21.4	3.09 ms	20.3

256 MiB	8.83 ms	28.5	8.60 ms	29.3
1 GiB	25.85 ms	38.9	24.24 ms	41.5
4 GiB	85.90 ms	46.9	85.38 ms	47.2
16 GiB	322.93 ms	49.9	332.52 ms	48.4

node5500+node5502 — gather

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	51.1 us	19.2	53.0 us	18.6
4 MiB	68.9 us	57.0	67.7 us	58.1
16 MiB	176.3 us	89.2	176.0 us	89.3
64 MiB	675.3 us	93.2	675.7 us	93.1
256 MiB	2.71 ms	93.0	2.71 ms	93.0
1 GiB	10.83 ms	92.9	10.83 ms	92.9
4 GiB	43.35 ms	92.9	43.34 ms	92.9
16 GiB	173.39 ms	92.9	173.39 ms	92.9

node5500+node5502 — scatter

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	363.8 us	2.7	248.5 us	4.0
4 MiB	354.4 us	11.1	185.7 us	21.2
16 MiB	167.5 us	93.9	163.0 us	96.5
64 MiB	325.3 us	193.4	328.4 us	191.6
256 MiB	1.17 ms	215.8	1.16 ms	216.3
1 GiB	4.26 ms	236.2	4.15 ms	242.3
4 GiB	12.66 ms	318.0	12.52 ms	321.7
16 GiB	49.26 ms	327.0	49.63 ms	324.5

node5501+node5502 — sendrecv

Message size	OOP time	OOP busbw	IP time	IP busbw

1 MiB	52.8 us	19.8	55.1 us	19.0
4 MiB	104.1 us	40.3	101.0 us	41.5
16 MiB	349.4 us	48.0	352.7 us	47.6
64 MiB	1.38 ms	48.8	1.37 ms	49.0
256 MiB	5.46 ms	49.2	5.45 ms	49.3
1 GiB	21.73 ms	49.4	21.75 ms	49.4
4 GiB	86.76 ms	49.5	86.76 ms	49.5
16 GiB	349.27 ms	49.2	347.87 ms	49.4

node5501+node5502 — all_reduce

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	498.3 us	4.0	517.4 us	3.8
4 MiB	1.61 ms	4.9	1.15 ms	6.9
16 MiB	1.15 ms	27.2	1.03 ms	30.7
64 MiB	2.50 ms	50.4	2.79 ms	45.1
256 MiB	5.08 ms	99.0	5.01 ms	100.4
1 GiB	13.21 ms	152.4	10.87 ms	185.1
4 GiB	37.36 ms	215.6	38.06 ms	211.6
16 GiB	140.44 ms	229.4	137.78 ms	233.8

node5501+node5502 — all_gather

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	586.5 us	1.7	591.2 us	1.7
4 MiB	511.8 us	7.7	503.6 us	7.8
16 MiB	550.5 us	28.6	474.8 us	33.1
64 MiB	1.33 ms	47.2	1.33 ms	47.2
256 MiB	1.46 ms	172.0	1.45 ms	173.3
1 GiB	3.21 ms	313.2	3.22 ms	313.0
4 GiB	12.05 ms	334.0	10.78 ms	373.5
16 GiB	43.20 ms	372.8	42.16 ms	382.0

node5501+node5502 — reduce_scatter

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	528.3 us	1.9	471.5 us	2.1
4 MiB	459.0 us	8.6	574.5 us	6.8
16 MiB	520.1 us	30.2	478.3 us	32.9
64 MiB	1.36 ms	46.4	1.35 ms	46.6
256 MiB	1.54 ms	163.5	1.51 ms	166.2
1 GiB	3.25 ms	309.6	3.27 ms	308.0
4 GiB	11.80 ms	341.3	11.60 ms	347.1
16 GiB	43.04 ms	374.2	43.07 ms	374.0

node5501+node5502 — reduce

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	388.3 us	2.7	364.1 us	2.9
4 MiB	413.0 us	10.2	102.1 us	41.1
16 MiB	181.6 us	92.4	177.6 us	94.5
64 MiB	390.7 us	171.8	389.3 us	172.4
256 MiB	1.29 ms	208.2	1.27 ms	211.2
1 GiB	4.57 ms	235.1	4.59 ms	233.7
4 GiB	13.65 ms	314.6	11.77 ms	364.8
16 GiB	45.37 ms	378.6	44.73 ms	384.1

node5501+node5502 — broadcast

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	462.6 us	2.3	411.3 us	2.5
4 MiB	119.1 us	35.2	116.8 us	35.9
16 MiB	184.3 us	91.0	181.7 us	92.3
64 MiB	382.5 us	175.4	386.0 us	173.8

256 MiB	1.24 ms	216.1	1.26 ms	213.7
1 GiB	4.33 ms	248.1	3.71 ms	289.7
4 GiB	13.42 ms	320.1	12.59 ms	341.0
16 GiB	47.93 ms	358.4	48.14 ms	356.9

node5501+node5502 — alltoall

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	857.8 us	1.1	714.7 us	1.4
4 MiB	538.5 us	7.3	474.5 us	8.3
16 MiB	1.05 ms	15.0	933.0 us	16.9
64 MiB	3.14 ms	20.0	3.05 ms	20.6
256 MiB	9.30 ms	27.1	9.13 ms	27.6
1 GiB	24.02 ms	41.9	25.57 ms	39.4
4 GiB	86.54 ms	46.5	86.31 ms	46.6
16 GiB	329.46 ms	48.9	332.49 ms	48.4

node5501+node5502 — gather

Message size	OOP time	OOP busbw	IP time	IP busbw
1 MiB	52.0 us	18.9	52.0 us	18.9
4 MiB	69.7 us	56.4	69.2 us	56.9
16 MiB	175.9 us	89.4	177.9 us	88.4
64 MiB	680.6 us	92.4	682.4 us	92.2
256 MiB	2.73 ms	92.1	2.73 ms	92.1
1 GiB	10.94 ms	92.0	10.94 ms	92.0
4 GiB	43.78 ms	92.0	43.78 ms	92.0
16 GiB	175.14 ms	92.0	175.14 ms	92.0

node5501+node5502 — scatter

Message size	OOP time	OOP busbw	IP time	IP busbw

1 MiB	392.8 us	2.5	315.5 us	3.1
4 MiB	340.1 us	11.6	238.1 us	16.5
16 MiB	202.1 us	77.8	207.0 us	76.0
64 MiB	327.9 us	191.9	336.9 us	186.7
256 MiB	977.6 us	257.4	971.4 us	259.1
1 GiB	3.52 ms	286.1	3.53 ms	285.1
4 GiB	12.78 ms	315.2	12.94 ms	311.1
16 GiB	47.76 ms	337.2	47.56 ms	338.6

OOP = out-of-place, IP = in-place.

Network fabric

The inter-node data path on the B200 nodes is **NDR (400 Gb/s)**:

NICs	Rate	Role
mlx5_4, 7, 8, 9, 10, 13, 14, 15	**400 Gb/s (4X NDR)**	8 GPU compute rails (active)
mlx5_0, 1, 2, 3	100 Gb/s (HDR100)	secondary (storage/mgmt)
mlx5_5, 6, 11, 12	down	unused

`nvidia_peermem` is loaded on both nodes, enabling GPUDirect RDMA so the NIC DMAs directly to/from GPU HBM over InfiniBand.